Análise e Complexidade de Algoritmos

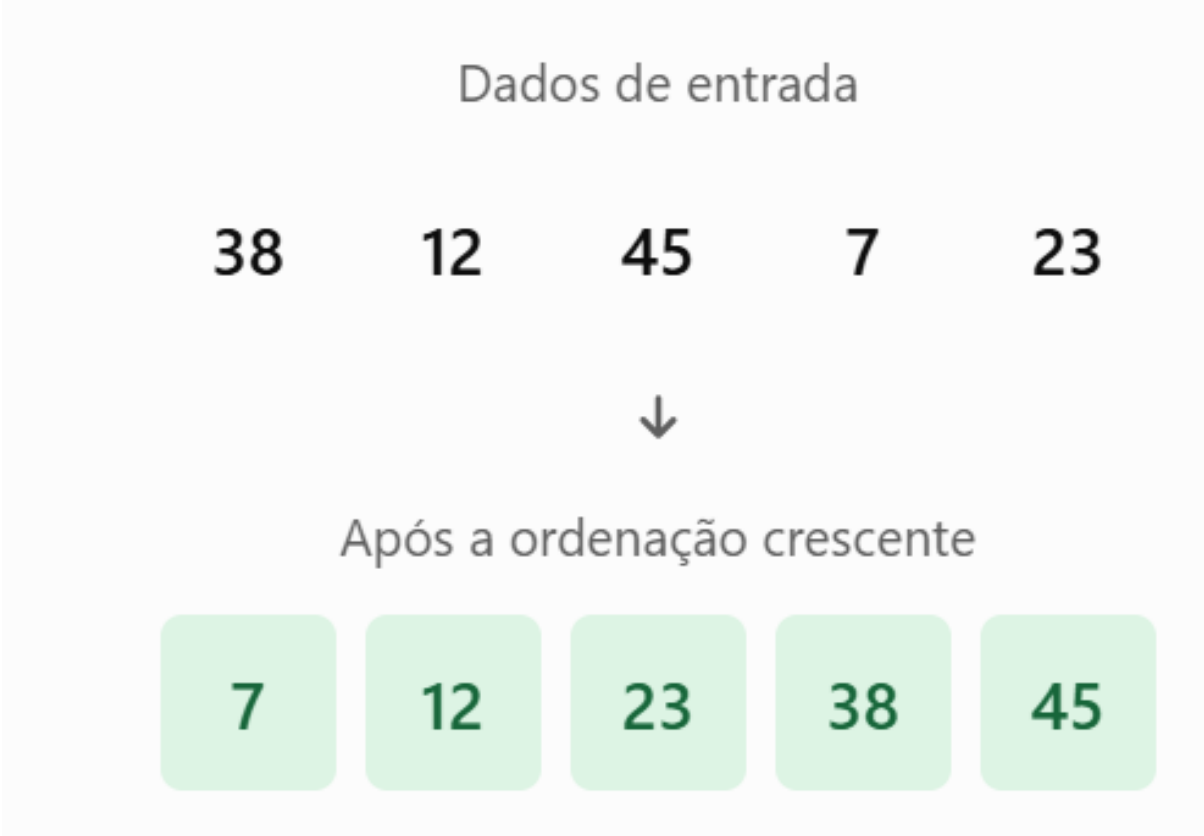
Aula 6

Análise de Algoritmos de Ordenação

Profª Drª Patricia Pereira

Ibmec
PROTAGONISTAS PARA O MUNDO

Por que estudar algoritmos de ordenação?



A ordenação consiste em reorganizar os elementos segundo um critério, por exemplo, ordem crescente ou decrescente.

Pergunta: se dois algoritmos produzem exatamente a mesma sequência ordenada, podemos afirmar que eles são igualmente eficientes?

Ibmec
PROTAGONISTAS PARA O MUNDO

Por que estudar algoritmos de ordenação?

Pergunta: se dois algoritmos produzem exatamente a mesma sequência ordenada, podemos afirmar que eles são igualmente eficientes?

Não. Eles podem diferir em:

- número de comparações;
- quantidade de movimentações dos elementos;
- tempo de execução;
- memória auxiliar utilizada;
- desempenho em entradas já ordenadas ou em ordem inversa.

Ibmec
PROTAGONISTAS PARA O MUNDO

O que significa ordenar corretamente?

Considere os dados: **[8, 3, 5, 3, 1]**
Uma ordenação crescente válida é: **[1, 3, 3, 5, 8]**

Para ordenar corretamente, o algoritmo deve atender a dois requisitos:

- Ordem: os elementos devem respeitar o critério estabelecido.
- Preservação dos elementos: nenhum elemento pode ser perdido, criado ou duplicado indevidamente.

Atenção: ordenar não significa eliminar duplicatas. Os dois valores ***3*** devem permanecer no resultado.

O que significa ordenar corretamente?

Considere os dados: **[8, 3, 5, 3, 1]**
Uma ordenação crescente válida é: **[1, 3, 3, 5, 8]**

Pergunta:
Se o algoritmo retornar **[1, 3, 5, 8]**, a sequência está em ordem crescente? A ordenação está correta?
Sim.
Mas é uma ordenação correta da entrada original?
Não.

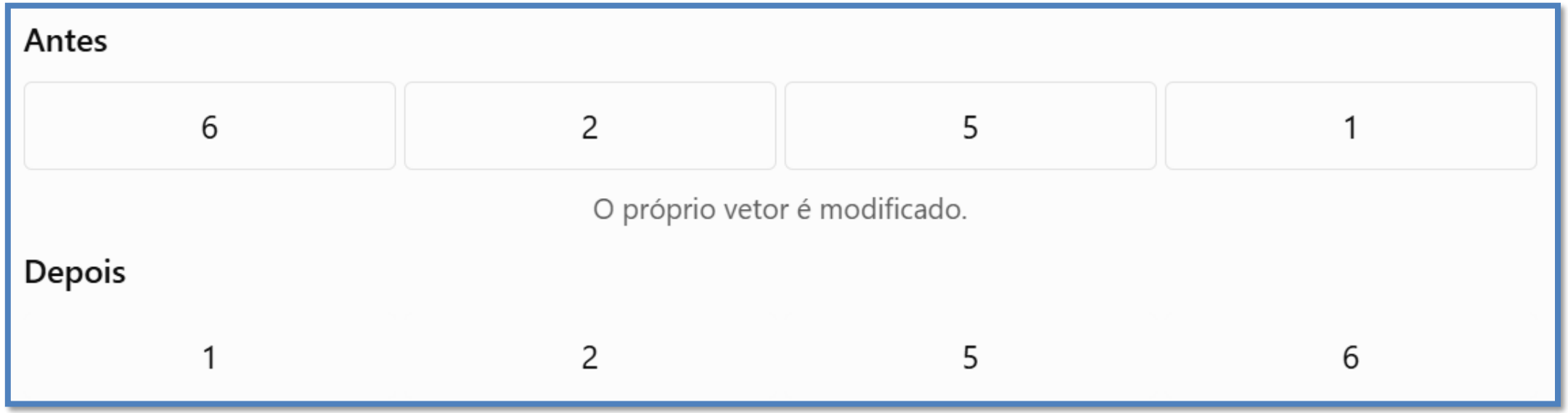
Como analisar um algoritmo de ordenação?

Critério	**Pergunta que devemos responder**
Tempo	Quanto cresce o trabalho com o tamanho da entrada?
Melhor caso	O que acontece quando a entrada favorece o algoritmo?
Pior caso	Qual é o maior custo para entradas de tamanho ***n***?
Caso médio	Qual é o custo esperado segundo um modelo de entradas?
Memória e estabilidade	Quanto espaço auxiliar é usado? A ordem relativa de iguais é preservada?

É importante separar os critérios: um algoritmo pode ter excelente tempo de execução e, ainda assim, precisar de memória adicional ou não preservar a ordem relativa de elementos iguais.

O que é um algoritmo in-place?

Um algoritmo ***in-place*** realiza a ordenação diretamente na estrutura original, utilizando uma quantidade pequena de memória auxiliar, normalmente O(1).



Ibmec
PROTAGONISTAS PARA O MUNDO

O que é um algoritmo in-place?

Em geral, **Bubble Sort, Selection Sort, Insertion Sort e Heap Sort** são classificados como ***in-place***.
Quick Sort também costuma ser classificado assim em suas implementações usuais, embora utilize espaço de pilha para as chamadas recursivas.
O ***Merge Sort*** tradicional para vetores normalmente utiliza um vetor auxiliar proporcional a *n*, portanto não é ***in-place*** no sentido usual.

Cuidado conceitual: ***in-place*** não significa necessariamente que o algoritmo não utiliza nenhuma memória adicional. É preciso distinguir espaço auxiliar para dados e espaço utilizado pela pilha de recursão.

Ibmec
PROTAGONISTAS PARA O MUNDO

O que é estabilidade?

Um algoritmo de ordenação é **estável** quando **preserva a ordem relativa** dos elementos que possuem a mesma chave de ordenação. Imagine que os registros sejam alunos ordenados por nota:

Entrada

Ana	Bruno	Caio
Nota 8	Nota 6	Nota 8

Ordenando pela nota em ordem crescente

Bruno	Ana	Caio
Nota 6	Nota 8	Nota 8

Ana continua antes de Caio, pois ambos têm nota 8 e Ana aparecia primeiro na entrada.

O que é estabilidade?

Por que isso importa?
Porque, em sistemas reais, os registros possuem outras informações além da chave usada para ordenar.
Exemplos:
ordenar funcionários por salário, produtos por preço ou alunos por nota.

Se dois registros tiverem a mesma chave, a estabilidade pode preservar uma ordenação anterior, como a ordem alfabética dos nomes.

Ibmec
PROTAGONISTAS PARA O MUNDO

Estabilidade não é o mesmo que *in-place*

Algoritmo	**In-place?**	**Estável?**
Bubble Sort	Sim	Sim, na versão usual
Selection Sort	Sim	Não, na versão usual
Insertion Sort	Sim	Sim
Merge Sort	Geralmente não, na versão tradicional para vetores	Sim, se implementado corretamente
Quick Sort	Geralmente sim	Não, na versão usual
Heap Sort	Sim	Não

Essas classificações dependem da implementação em alguns casos.

Por exemplo, o Merge Sort pode ser implementado com estratégias especiais de memória, e um Quick Sort estável exige uma implementação diferente da usual.

Pergunta

Qual propriedade você priorizaria ao ordenar uma lista de registros que já foi ordenada por outro critério?

Resposta esperada: estabilidade, se for necessário preservar a ordem anterior entre registros com a mesma chave.

Algoritmos quadráticos

Bubble Sort, Selection Sort e Insertion Sort são úteis para compreender os fundamentos da ordenação.

Em suas versões usuais, apresentam custo de $O(n^2)$ em situações relevantes, mas não se comportam da mesma forma em todas as entradas.

Vamos usar o vetor: **[5, 2, 4, 1]**
A ideia é acompanhar como cada algoritmo modifica a sequência, e não apenas memorizar seu resultado.

Ibmec
PROTAGONISTAS PARA O MUNDO

Bubble Sort: ideia

O Bubble Sort compara elementos adjacentes e troca suas posições quando estão fora de ordem.

Exemplo: **[5, 2, 4, 1]**
Primeira comparação: 5 e 2. (trocar)
[5, 2, 4, 1] ➔ [2, 5, 4, 1]
Segunda comparação: 5 e 4. (trocar)
[2, 5, 4, 1] ➔ [2, 4, 5, 1]
Terceira comparação: 5 e 1. (trocar)
[2, 4, 5, 1] ➔ [2, 4, 1, 5]
Ao final dessa passagem, o maior elemento está na última posição.
[2, 4, 1, 5]

Ibmec
PROTAGONISTAS PARA O MUNDO

Bubble Sort: ideia

Repetindo essa ideia para o subvetor
[2, 4, 1, 5]

Primeira comparação: 2 e 4. (não trocar)
[2, 4, 1, 5]

Segunda comparação: 4 e 1. (trocar)
[2, 4, 1, 5]➔ [2, 1, 4, 5]

Ao final dessa passagem, o maior elemento do subvetor está na última posição correta.
[2, 1, 4, 5]

Ibmec
PROTAGONISTAS PARA O MUNDO

Bubble Sort: ideia

Repetindo essa ideia para o subvetor
[2, 1, 4, 5]

Primeira comparação: 2 e 1. (trocar)
[2, 1, 4, 5] ➔ **[1, 2, 4, 5]**

Ao final dessa passagem, o vetor está ordenado

[1, 2, 4, 5]

Ibmec
PROTAGONISTAS PARA O MUNDO

Bubble Sort: implementação

```
def bubble_sort(v):
    n = len(v)

    for i in range(n - 1):
        for j in range(n - 1 - i):
            if v[j] > v[j + 1]:
                v[j], v[j + 1] = v[j + 1], v[j]

    return v
```

o laço externo

controla as passagens

Ibmec
PROTAGONISTAS PARA O MUNDO

Bubble Sort: implementação

```
def bubble_sort(v):
    n = len(v)

    for i in range(n - 1):
        for j in range(n - 1 - i):
            if v[j] > v[j + 1]:
                v[j], v[j + 1] = v[j + 1], v[j]

    return v
```

o laço interno compara elementos adjacentes

cada troca corrige uma inversão local;
o limite do laço interno diminui,
pois os maiores elementos já estão nas posições finais.

Ibmec
PROTAGONISTAS PARA O MUNDO

Bubble Sort: implementação

```
def bubble_sort(v):
    n = len(v)

    for i in range(n - 1):
        for j in range(n - 1 - i):
            if v[j] > v[j + 1]:
                v[j], v[j + 1] = v[j + 1], v[j]

    return v
```

Pergunta: por que a condição utiliza **>** em vez de **>=**?

Porque não trocar elementos iguais contribui para preservar a estabilidade.

Ibmec
PROTAGONISTAS PARA O MUNDO

Bubble Sort: complexidade

Na implementação anterior, o algoritmo sempre executa as passagens previstas, mesmo quando o vetor já está ordenado.

- Melhor caso: $\Theta(n^2)$.
- Caso médio: $\Theta(n^2)$.
- Pior caso: $\Theta(n^2)$.
- Espaço auxiliar: $\Theta(1)$.
- *In-place*: sim.
- Estável: sim, com a condição de troca apresentada.

Observação importante: uma versão otimizada, que interrompe o algoritmo quando uma passagem não realiza nenhuma troca, tem melhor caso ***Θ(n)***.
É necessário analisar o código concreto, não apenas o nome do algoritmo.

Ibmec
PROTAGONISTAS PARA O MUNDO

Selection Sort: ideia

O Selection Sort procura o menor elemento da parte ainda não ordenada e o coloca na próxima posição correta.

Entrada: **[5, 2, 4, 1]**
Primeira passagem: o menor é **1**.
Troca com o primeiro elemento: **[1, 2, 4, 5]**
Segunda passagem: o menor dos elementos restantes é **2**.
Ele já está na posição correta.
O algoritmo continua selecionando o menor elemento restante até terminar.
Diferentemente do **Bubble Sort**, o **Selection Sort** não precisa trocar elementos a cada comparação.

Ibmec
PROTAGONISTAS PARA O MUNDO

Selection Sort: implementação

```
def selection_sort(v):
    n = len(v)

    for i in range(n - 1):
        menor = i

        for j in range(i + 1, n):
            if v[j] < v[menor]:
                menor = j

        if menor != i:
            v[i], v[menor] = v[menor], v[i]

    return v
```

O que acontece em cada passagem?

1. Assume-se que a posição atual contém o menor elemento.
2. Percorre-se a parte não ordenada.
3. Atualiza-se o índice do menor elemento encontrado.
4. Realiza-se, no máximo, uma troca para posicioná-lo.

Ibmec
PROTAGONISTAS PARA O MUNDO

Selection Sort: complexidade

O algoritmo precisa examinar a parte não ordenada para encontrar o menor elemento, mesmo que o vetor já esteja ordenado.

- Melhor caso: $\Theta(n^2)$.
- Caso médio: $\Theta(n^2)$.
- Pior caso: $\Theta(n^2)$.
- Espaço auxiliar: $\Theta(1)$.
- *In-place*: sim.
- Estável: não, na versão usual.

Uma vantagem é que o número de trocas é limitado a **O(n)**.

Isso pode ser relevante quando movimentar elementos é caro.

Confirme a instabilidade com o exemplo 3 1 3 5 2 3

Ibmec
PROTAGONISTAS PARA O MUNDO

Selection Sort: pergunta

Se o vetor já estiver ordenado, o **Selection Sort** consegue evitar as comparações?

Não.

Ele ainda precisa procurar o menor elemento em cada

parte restante

Ibmec
PROTAGONISTAS PARA O MUNDO

Insertion Sort: ideia

O Insertion Sort constrói gradualmente uma região ordenada, inserindo cada novo elemento na posição correta.
É semelhante à maneira como organizamos cartas na mão.

Entrada: **[5, 2, 4, 1]**

Inserindo **5**: **[5, 2, 4, 1]** Primeiro elemento considerado ordenado
Inserindo **2**: **[2, 5, 4, 1]**
Inserindo **4**: **[2, 4, 5, 1]**
Inserindo **1**: **[1, 2, 4, 5]**

Observe que o algoritmo desloca os elementos maiores para abrir espaço para o elemento que será inserido.

Ibmec
PROTAGONISTAS PARA O MUNDO

Insertion Sort: implementação e análise

```
def insertion_sort(v):
    for i in range(1, len(v)):
        chave = v[i]
        j = i - 1

        while j >= 0 and v[j] > chave:
            v[j + 1] = v[j]
            j -= 1

        v[j + 1] = chave

    return v
```

Complexidades:

Melhor caso: **Θ(n)** , quando o vetor já está ordenado.
Caso médio: **Θ(n^2)**.
Pior caso: **Θ(n^2)**, por exemplo, quando os dados estão em ordem decrescente.
Espaço auxiliar: **Θ(1)**
In-place: sim.
Estável: sim, na implementação apresentada.

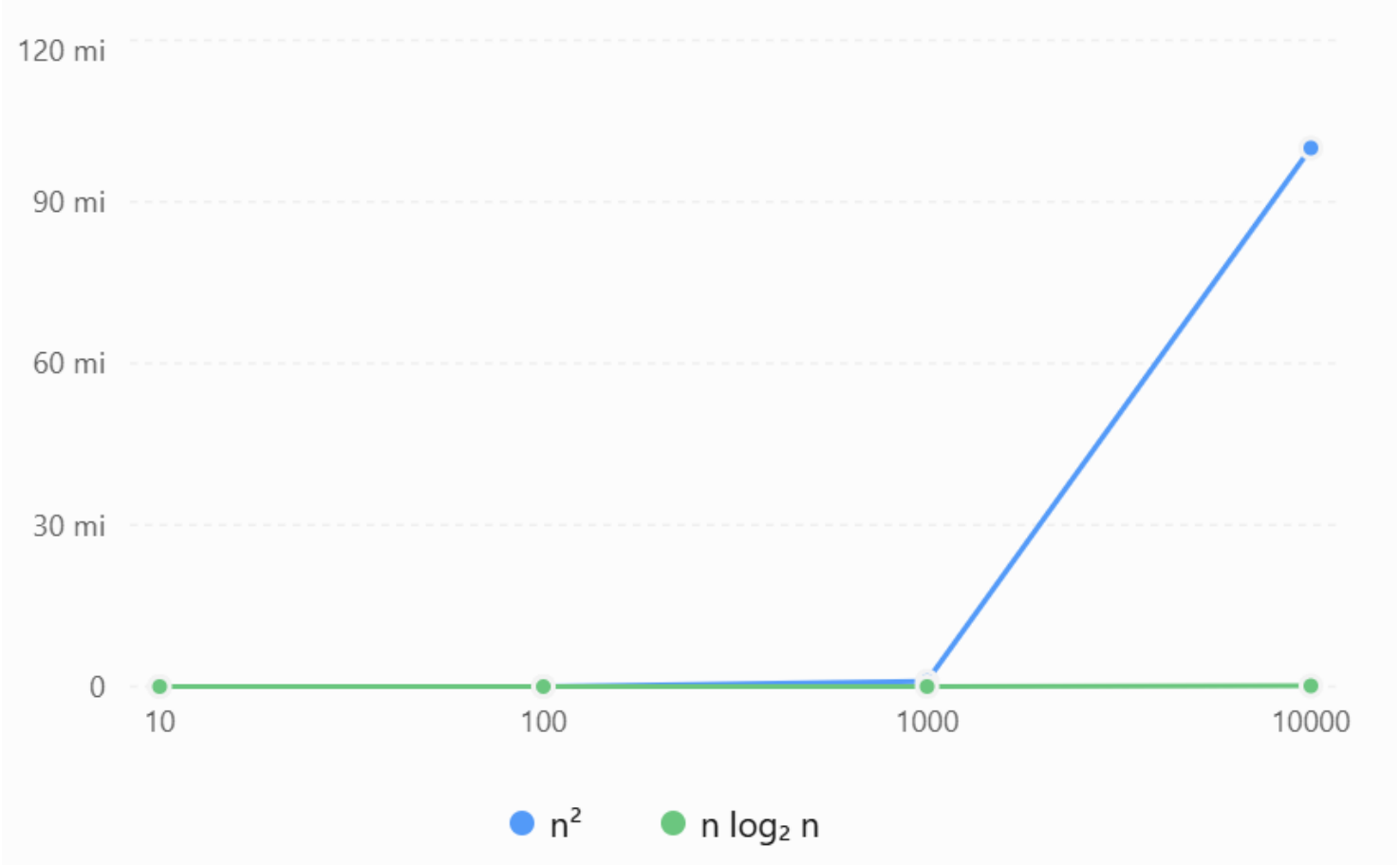
Observação: embora os três algoritmos tenham custo quadrático em situações relevantes, o **Insertion Sort** pode ser bastante eficiente para entradas pequenas ou quase ordenadas.

Algoritmos mais eficientes

Por que precisamos de outros algoritmos?

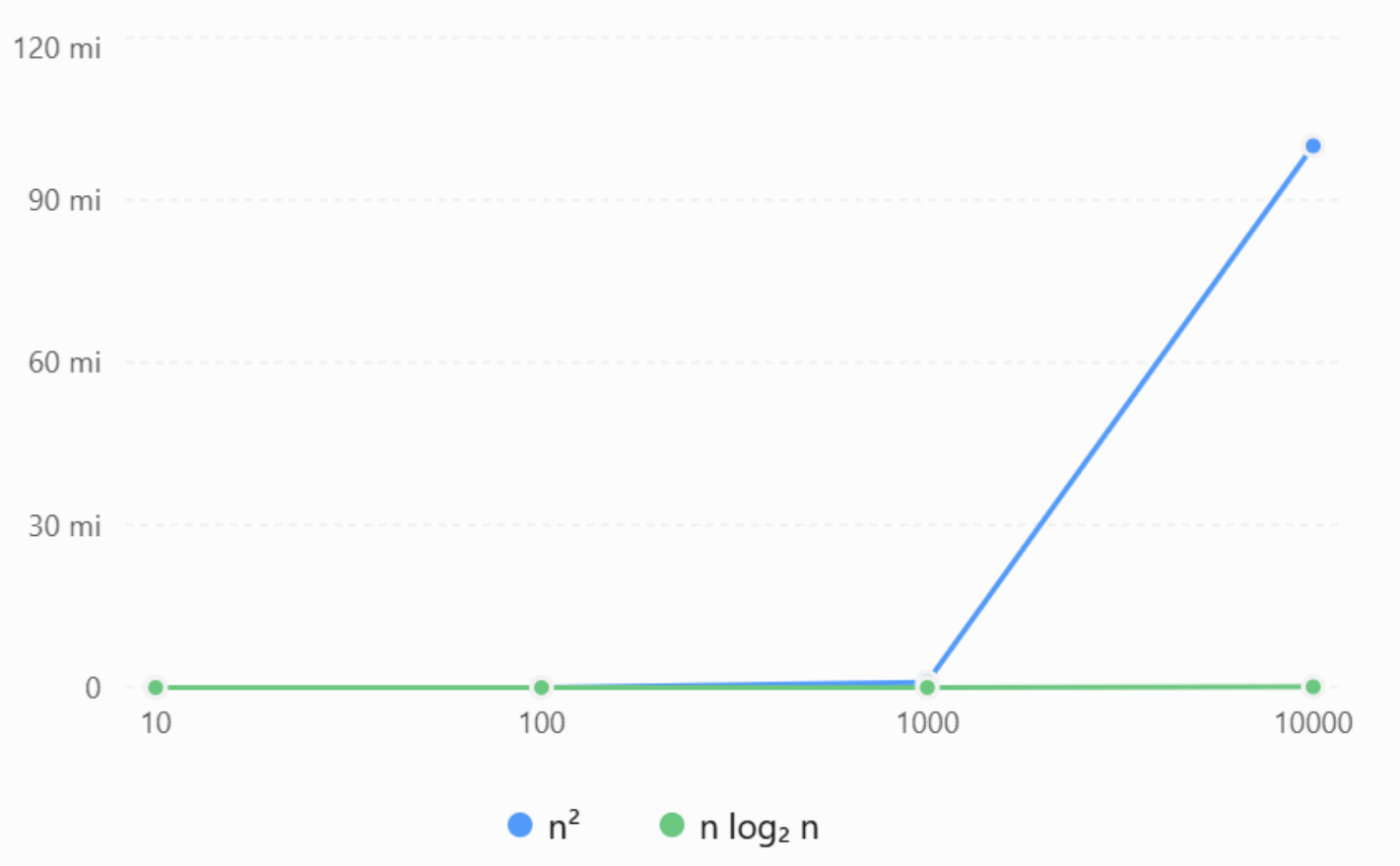
Imagine ordenar um milhão de elementos.

Comparação ilustrativa das funções n^2 e $n \log_2 n$.



Algoritmos mais eficientes

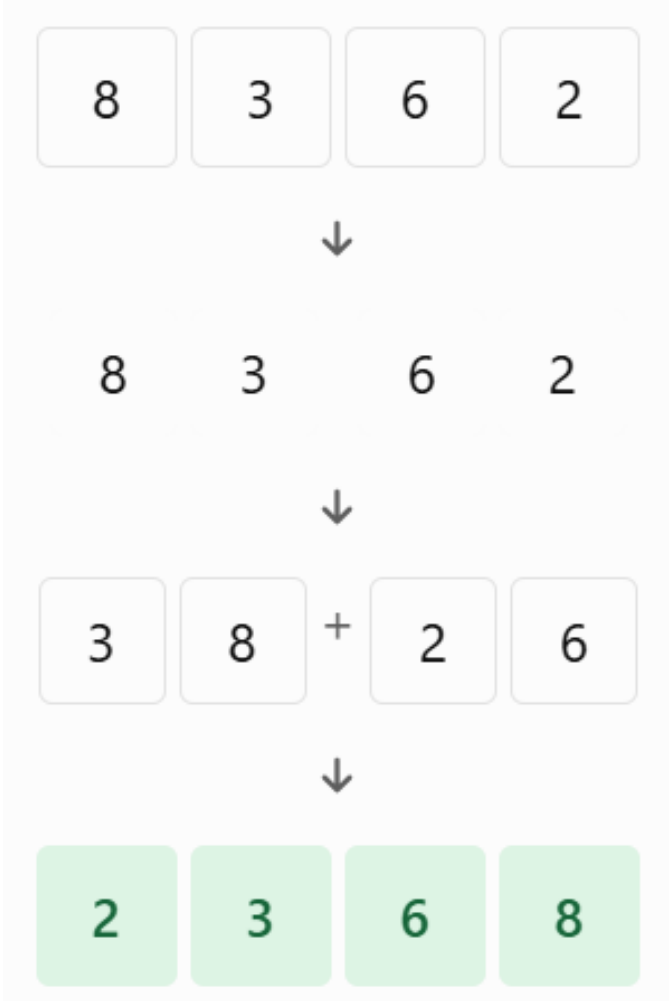
O gráfico ilustra o crescimento das funções, e não tempos de execução medidos. Na prática, constantes, implementação e características dos dados também influenciam o desempenho.



Mensagem central: para entradas grandes, a diferença entre $\mathbf{n^2}$ e $\mathbf{n\log_2 n}$ pode ser enorme.

Merge Sort: ideia

O Merge Sort utiliza a estratégia de dividir para conquistar, conectando-se diretamente à aula anterior sobre recorrências.



Seu funcionamento pode ser organizado em três etapas:

1. Dividir o vetor em duas partes.
2. Ordenar recursivamente cada parte.
3. Intercalar as duas partes ordenadas.

A divisão continua até chegarmos a subvetores de um único elemento. Depois, as partes são intercaladas em ordem.

Merge Sort: implementação

```
if len(v) <= 1:
    return v

meio = len(v) // 2

esquerda = merge_sort(v[:meio])
direita = merge_sort(v[meio:])

resultado = []
i = j = 0

while i < len(esquerda) and j < len(direita):
    if esquerda[i] <= direita[j]:
        resultado.append(esquerda[i])
        i += 1
    else:
        resultado.append(direita[j])
        j += 1

resultado.extend(esquerda[i:])
```

Caso base: vetor com zero ou um elemento.

Duas chamadas recursivas: uma para cada metade.
Intercalação: percorre as duas partes ordenadas.

O resultado é construído em uma nova lista.

Ibmec
PROTAGONISTAS PARA O MUNDO

Merge Sort: implementação

```
if len(v) <= 1:
    return v

meio = len(v) // 2

esquerda = merge_sort(v[:meio])
direita = merge_sort(v[meio:])

resultado = []
i = j = 0

while i < len(esquerda) and j < len(direita):
    if esquerda[i] <= direita[j]:
        resultado.append(esquerda[i])
        i += 1
    else:
        resultado.append(direita[j])
        j += 1

resultado.extend(esquerda[i:])
```

A utilização de **<=** para escolher o elemento da esquerda quando há empate ajuda a preservar a estabilidade.

Observação: esta implementação cria novas listas por meio de fatiamento e de resultado. Portanto, utiliza memória auxiliar proporcional ao tamanho da entrada.

Ibmec
PROTAGONISTAS PARA O MUNDO

Merge Sort: construindo a recorrência

Considere ***n*** como o número de elementos.

A cada chamada:

- são resolvidos dois subproblemas de tamanho aproximado **n/2**;
- o trabalho de intercalação é linear, **Θ(n)**.

$$T(n) = 2T(n/2) + \Theta(n)$$

Merge Sort: construindo a recorrência

$$T(n) = 2T(n/2) + \Theta(n)$$

Pela árvore de recursão, cada nível possui custo total proporcional a ***n***, e a profundidade é proporcional a ***log n***.

$$T(n) = \Theta(n \log n)$$

O resultado vale para o melhor caso, o caso médio e o pior caso do **Merge Sort** tradicional.

Espaço auxiliar para esta implementação: **Θ(n)**.

Ibmec
PROTAGONISTAS PARA O MUNDO

Quick Sort: ideia

O **Quick Sort** também utiliza a estratégia de dividir para conquistar, mas funciona de forma diferente.

Ele escolhe um pivô e particiona os elementos em grupos:

- elementos menores que o pivô;
- pivô em sua posição final de partição;
- elementos maiores que o pivô.

Depois, ordena recursivamente as regiões à esquerda e à direita.

Quick Sort: ideia

Exemplo com pivô 5: [8, 3, 6, 2, 5, 7, 1]

Uma partição possível é: [3, 2, 1] 5 [8, 6, 7]

A disposição interna dos grupos pode variar conforme a implementação.

Diferença importante:
No **Merge Sort**, dividimos o vetor em metades e depois intercalamos.
No **Quick Sort**, a partição é realizada em torno do pivô e, após ordenar as partes, não é necessária uma intercalação final

Ibmec
PROTAGONISTAS PARA O MUNDO

Quick Sort: implementação didática

```python
def quick_sort(v):
    if len(v) <= 1:
        return v

    pivo = v[len(v) // 2]

    menores = [x for x in v if x < pivo]
    iguais = [x for x in v if x == pivo]
    maiores = [x for x in v if x > pivo]

    return (
        quick_sort(menores)
        + iguais
        + quick_sort(maiores)
    )
```

Essa versão usa um novo vetor para simplificar a compreensão da partição.

Mantém todos os elementos iguais ao pivô em um grupo e facilita o tratamento de duplicatas.

Atenção: essa não é uma implementação ***in-place*** e seu uso de listas auxiliares altera o perfil de memória em relação ao ***Quick Sort*** clássico.

Quick Sort: melhor, médio e pior caso

O desempenho depende de como o pivô divide os elementos.

Melhores divisões

As partições têm tamanhos aproximadamente equilibrados.

Altura aproximada: $\log n$

Θ(n log n)

Pior caso

Uma partição tem quase todos os elementos e a outra é vazia.

Profundidade: proporcional a n

Θ(n^2)

Quick Sort: melhor, médio e pior caso

Para o Quick Sort clássico:

- Melhor caso: $\Theta(n \log n)$.
- Caso médio: $\Theta(n \log n)$, sob hipóteses usuais de entradas e escolha do pivô.
- Pior caso: $\Theta(n^2)$.
- Versão usual: *in-place* em relação ao vetor, mas utiliza pilha de recursão.
- Estabilidade: não, na versão usual.

Escolher o pivô aleatoriamente ou usar estratégias melhores de seleção reduz a probabilidade de partições muito ruins, mas não elimina o pior caso teórico do Quick Sort clássico.

Quick Sort: implementação clássica (in-place)

```python
def quick_sort(v, inicio, fim):
    if inicio < fim:
        pos_pivo = particionar(v, inicio, fim)

        quick_sort(v, inicio, pos_pivo - 1)
        quick_sort(v, pos_pivo + 1, fim)

    return v
```

```python
def particionar(v, inicio, fim):
    pivo = v[fim]
    i = inicio - 1

    for j in range(inicio, fim):
        if v[j] <= pivo:
            i += 1
            v[i], v[j] = v[j], v[i]

    v[i + 1], v[fim] = v[fim], v[i + 1]

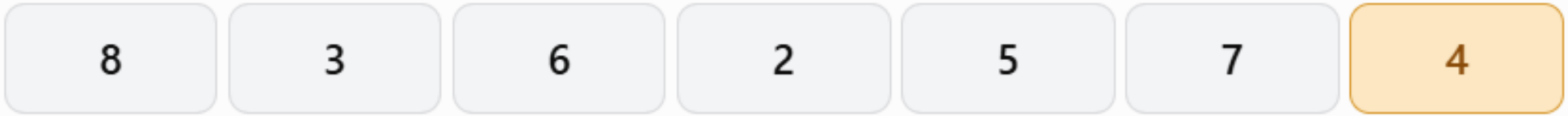
    return i + 1
```

Ibmec
PROTAGONISTAS PARA O MUNDO

Como funciona a implementação clássica?

1. Escolha do pivô

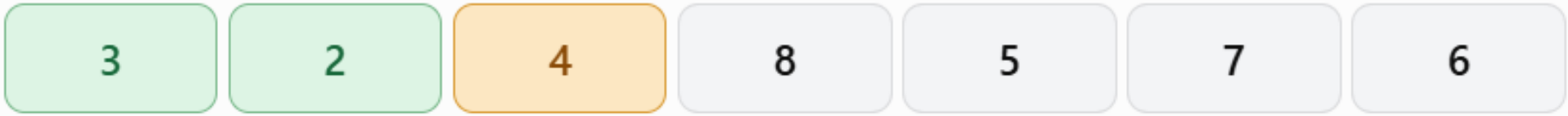
O último elemento da região é escolhido como pivô.



Pivô = 4

2. Particionamento

O algoritmo coloca os elementos menores ou iguais ao pivô à esquerda e os maiores à direita. Ao final, o pivô ocupa sua posição definitiva.



Menores que o pivô Pivô Maiores que o pivô

O pivô 4 fica na terceira posição, considerando a contagem a partir de 1. As duas regiões serão ordenadas recursivamente.

Como funciona a implementação clássica?

3. Recursão

O algoritmo chama a si mesmo para ordenar as duas regiões, sem incluir novamente o pivô, pois ele já está na posição definitiva.

Partição à esquerda

3	2

Partição à direita

8	5	7	6

Resultado final

2	3	4	5	6	7	8

Comparação com a versão didática

Característica	Quick Sort didático	Quick Sort clássico
Estratégia	Cria listas de menores, iguais e maiores	Particiona o vetor original
Pivô	Elemento central por índice	Último elemento da região
Memória auxiliar	Listas adicionais	Não cria listas auxiliares proporcionais à entrada
In-place	Não	Sim, na organização do vetor
Recursividade	Duas chamadas sobre listas	Duas chamadas sobre intervalos do vetor
Estabilidade	Não é garantida	Não
Melhor caso	$\Theta(n \log n)$	$\Theta(n \log n)$
Caso médio	Em condições usuais, $\Theta(n \log n)$	Em condições usuais, $\Theta(n \log n)$
Pior caso	$\Theta(n^2)$	$\Theta(n^2)$

Observação sobre memória: a versão clássica evita criar listas auxiliares proporcionais à entrada, mas utiliza a pilha de recursão. Seu espaço de pilha é tipicamente **Θ(log n)** no caso equilibrado e pode chegar a **Θ(n)** no pior caso.

Heap Sort: ideia

O **Heap Sort** utiliza uma estrutura chamada ***heap***, que permite acessar eficientemente o maior elemento (em uma ordenação crescente, usando um heap máximo).

O algoritmo:

1. Constrói um heap máximo.
2. Troca a raiz, que contém o maior elemento, com o último elemento da região não ordenada.
3. Reduz a região do heap.
4. Restaura a propriedade do heap.
5. Repete até terminar.

A cada etapa, o maior elemento restante é colocado na posição final.

Ibmec
PROTAGONISTAS PARA O MUNDO

Heap Sort: exemplo passo a passo

Vamos ordenar o vetor em ordem crescente utilizando o Heap Sort, que transforma o vetor em um heap máximo e, depois, coloca repetidamente o maior elemento na posição final.

Vetor inicial:

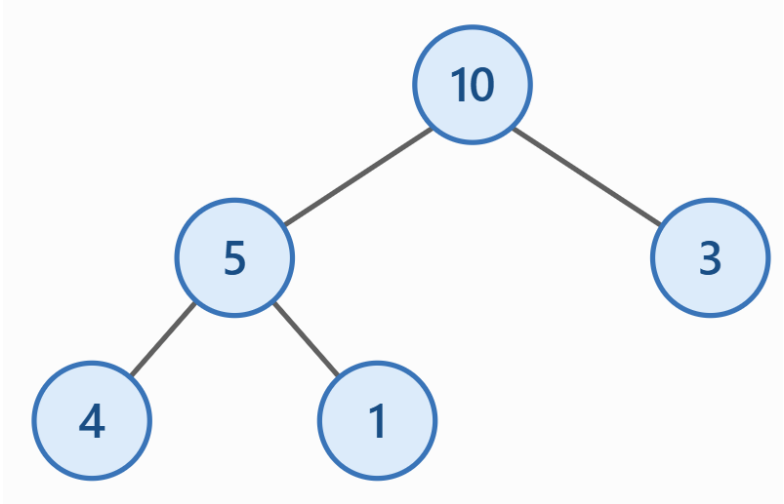
$$[\mathbf{4}, \mathbf{10}, \mathbf{3}, \mathbf{5}, \mathbf{1}]$$

Ibmec
PROTAGONISTAS PARA O MUNDO

Heap Sort: exemplo passo a passo

1. Construindo o heap máximo

Um heap máximo é uma árvore binária em que cada pai possui valor maior ou igual ao de seus filhos.



Representação da árvore após a construção do heap máximo.

Vetor que representa o heap:

[10, 5, 3, 4, 1]

Observe que o maior elemento, 10, está na raiz. O heap não precisa estar totalmente ordenado: ele precisa respeitar a relação entre pais e filhos.

Ibmec
PROTAGONISTAS PARA O MUNDO

Heap Sort: exemplo passo a passo

2. Colocando o maior elemento na posição final

Trocamos a raiz (**10**) com o último elemento da região do heap.

Após posicionar o 10

5	4	3	1	10

Após posicionar o 5

4	1	3	5	10

Após posicionar o 4

3	1	4	5	10

Resultado final

1	3	4	5	10

As posições em verde representam elementos que já estão na região ordenada definitiva.

Heap Sort: implementação

```python
def heapify(v, n, i):
    maior = i
    esquerda = 2 * i + 1
    direita = 2 * i + 2

    if esquerda < n and v[esquerda] > v[maior]:
        maior = esquerda

    if direita < n and v[direita] > v[maior]:
        maior = direita

    if maior != i:
        v[i], v[maior] = v[maior], v[i]
        heapify(v, n, maior)
```

```python
def heap_sort(v):
    n = len(v)

    # Construção do heap máximo
    for i in range(n // 2 - 1, -1, -1):
        heapify(v, n, i)

    # Extração sucessiva do maior elemento
    for i in range(n - 1, 0, -1):
        v[0], v[i] = v[i], v[0]
        heapify(v, i, 0)

    return v
```

Ibmec
PROTAGONISTAS PARA O MUNDO

Heap Sort: Análise da complexidade

Etapa	Complexidade
Construção do heap máximo	$\Theta(n)$
Restauração após cada extração	$O(\log n)$
Extrações sucessivas	$\Theta(n \log n)$ no total
Tempo total	$\Theta(n \log n)$
Espaço auxiliar	$O(1)$ para a versão recursiva acima?

Ibmec
PROTAGONISTAS PARA O MUNDO

Heap Sort: complexidade

Para o Heap Sort tradicional:

- Construção inicial do heap: $\Theta(n)$.
- Cada remoção/restauração do heap: $O(\log n)$.
- São realizadas aproximadamente n extrações.

Logo, o custo total é:

$$\Theta(n) + \Theta(n \log n) = \boxed{\Theta(n \log n)}$$

Heap Sort: complexidade

Características:

- Melhor caso: $\Theta(n \log n)$.
- Caso médio: $\Theta(n \log n)$.
- Pior caso: $\Theta(n \log n)$.
- Espaço auxiliar: $\Theta(1)$ na implementação iterativa convencional.
- *In-place*: sim.
- Estável: não, na versão usual.

Observação: o **Heap Sort** oferece uma garantia de tempo no pior caso de O(nlog n) e baixo uso de espaço auxiliar.

Ibmec
PROTAGONISTAS PARA O MUNDO

Heap Sort: observação importante

O código utiliza ***heapify*** recursivamente.

Embora o ***Heap Sort*** seja ***in-place*** quanto ao vetor, essa implementação pode utilizar ***O(log n)*** de espaço na pilha durante a restauração do ***heap***.

Para obter espaço auxiliar total *O(1)*, seria necessário implementar ***heapify*** iterativamente.

O ***Heap Sort*** tradicional apresenta ***Θ(nlog n)*** no melhor, médio e pior caso, é ***in-place*** no sentido usual e não é estável.

Ibmec
PROTAGONISTAS PARA O MUNDO

Heap Sort: Pergunta

Por que construir o heap custa ***Θ(n)*** , mas o algoritmo completo custa ***Θ(nlog n)*** ?

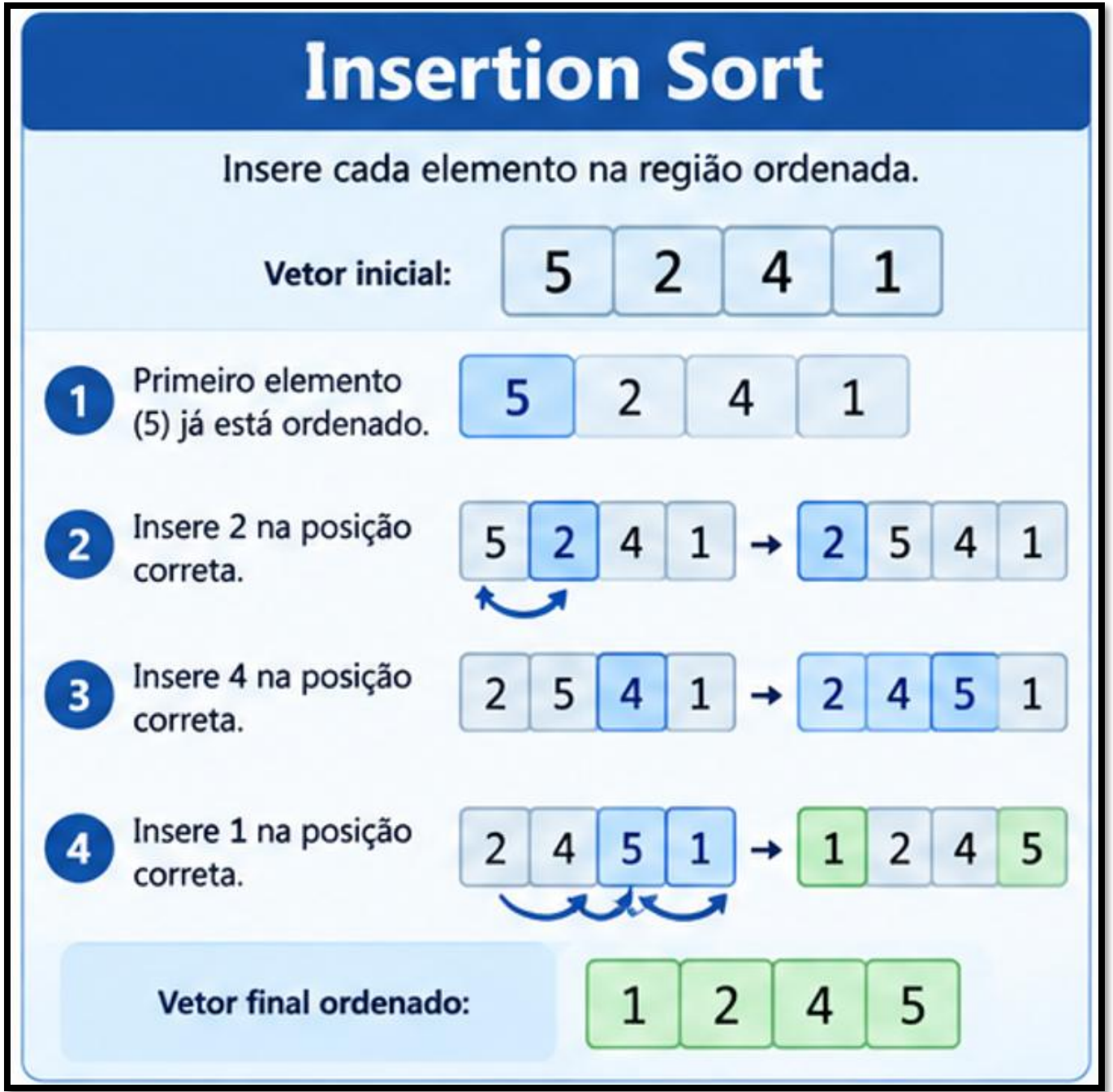
Resposta esperada: porque a construção inicial do heap é linear, mas, depois, realizamos aproximadamente ***n*** extrações, cada uma seguida de uma restauração que custa até **O(log n).**

Ibmec
PROTAGONISTAS PARA O MUNDO

Comparação visual das estratégias



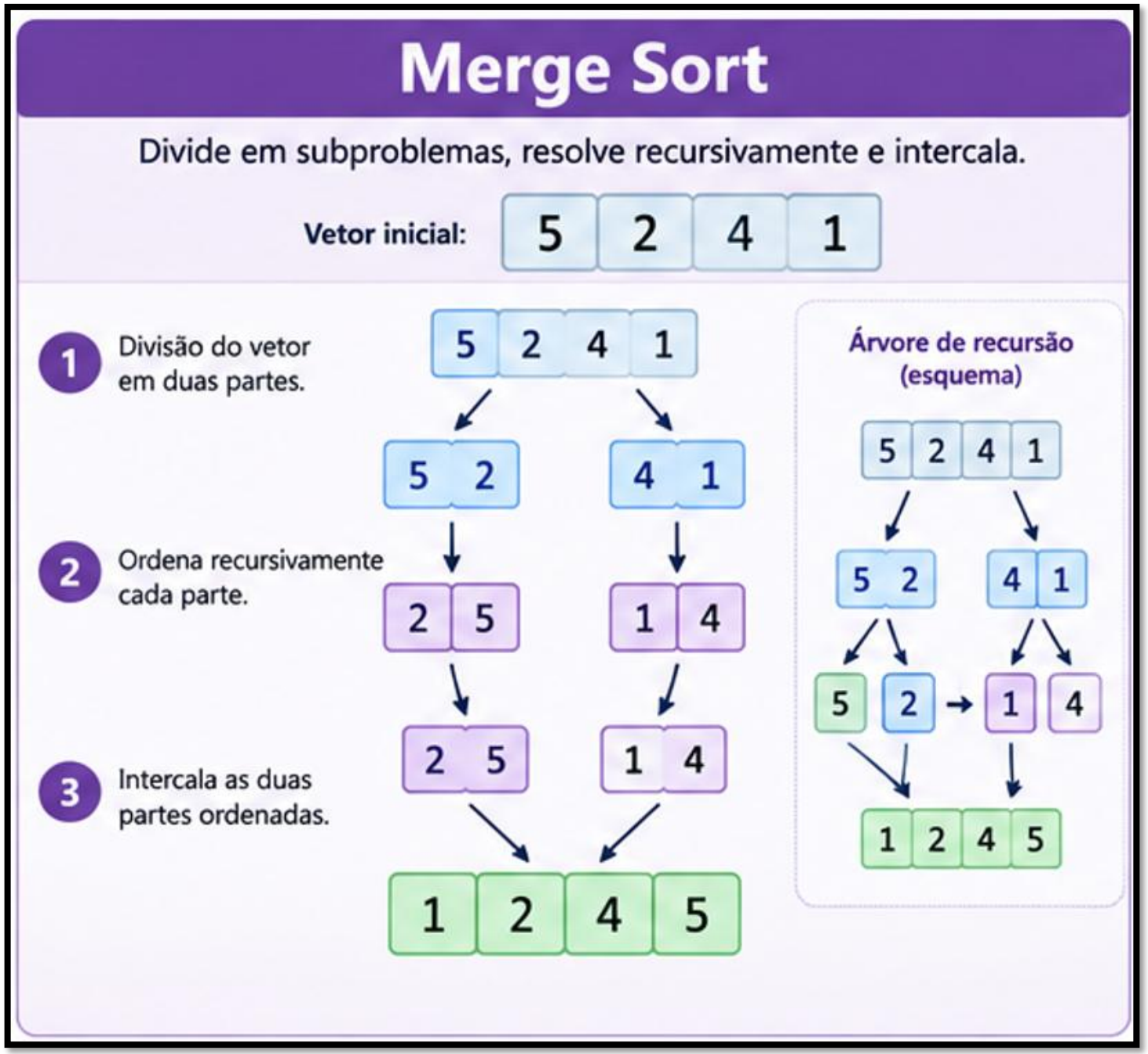
Comparação visual das estratégias



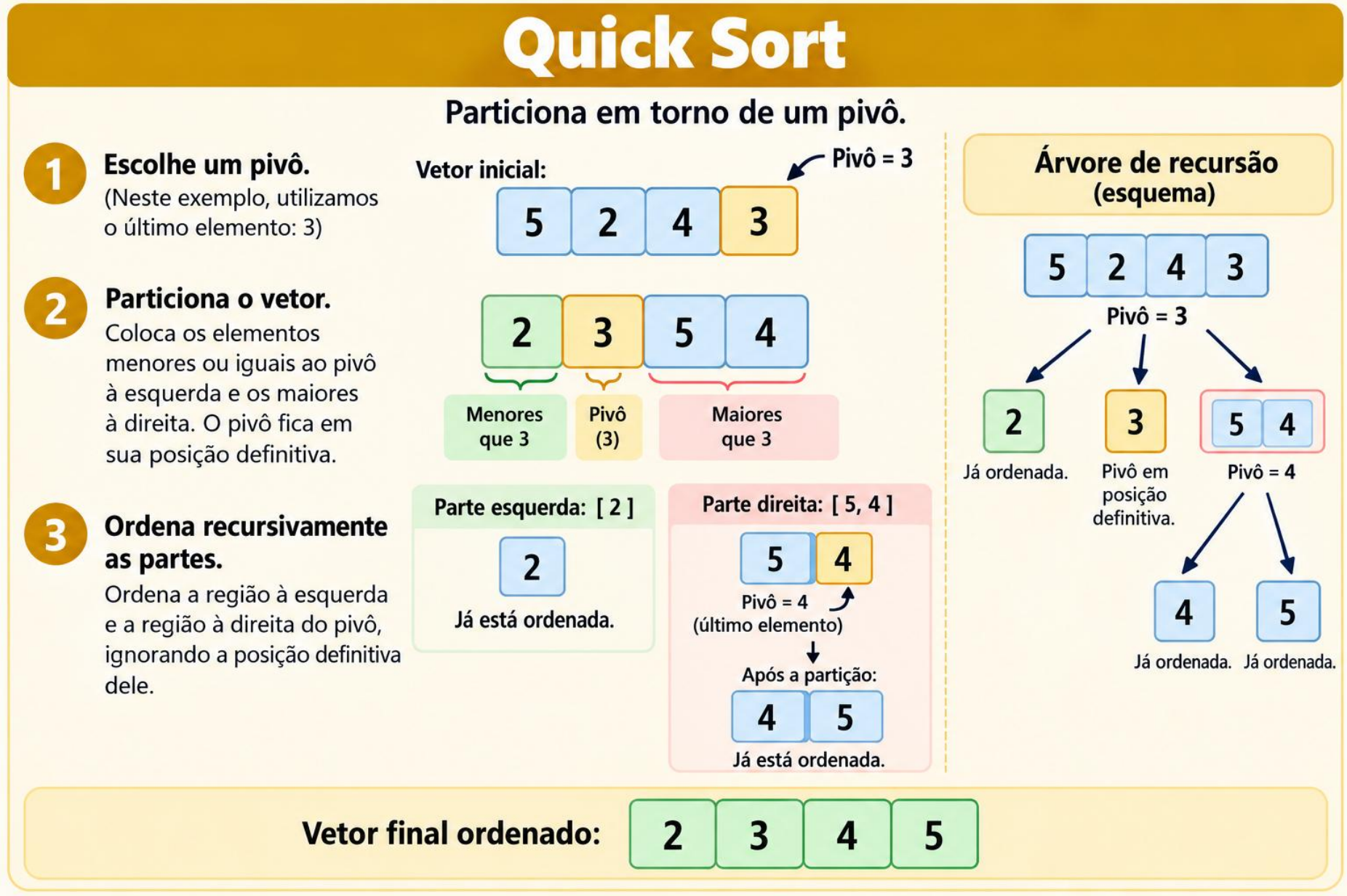
Comparação visual das estratégias



Comparação visual das estratégias



Comparação visual das estratégias



Comparação visual das estratégias

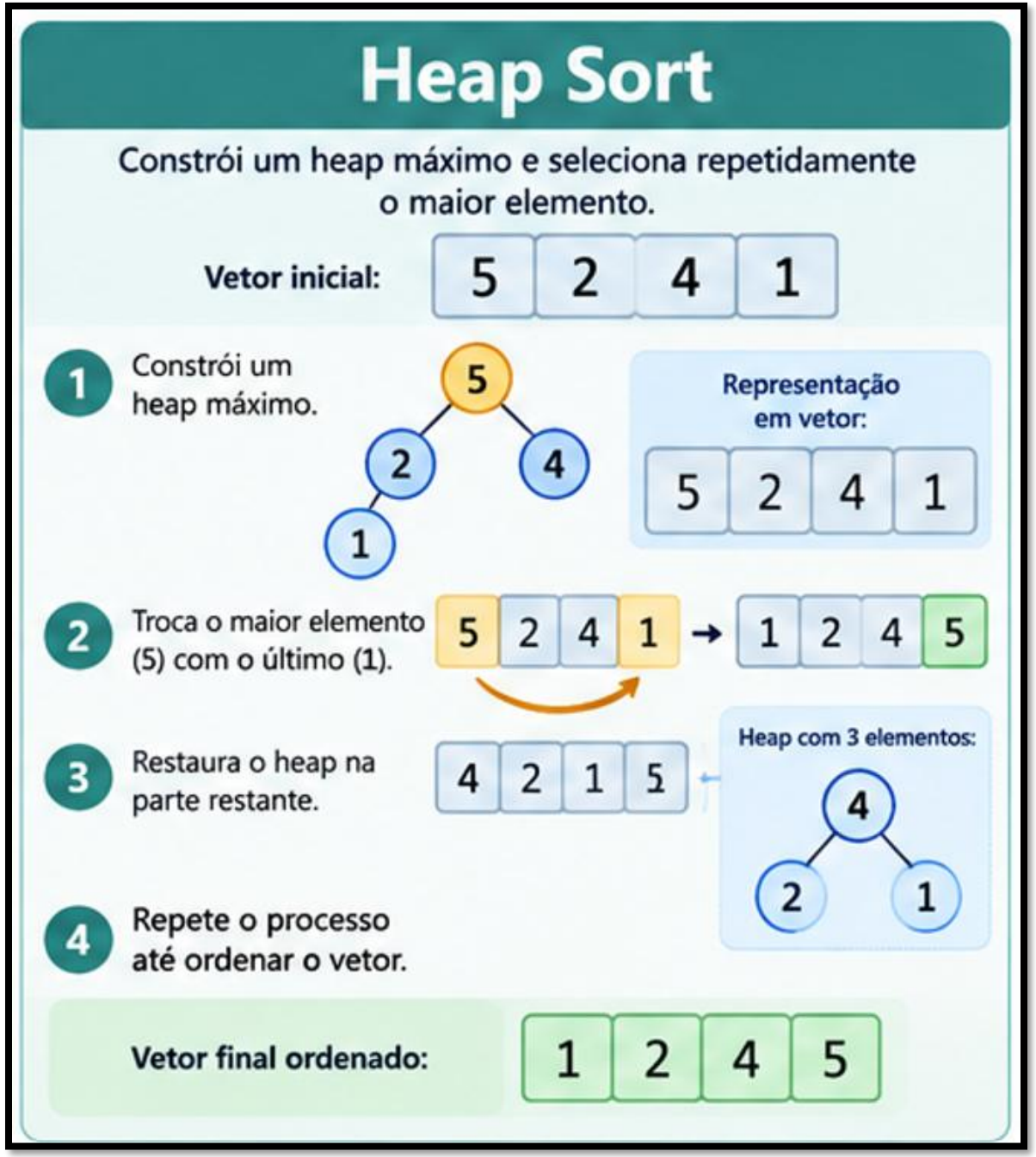


Tabela comparativa de complexidade

Algoritmo	Melhor caso	Caso médio	Pior caso
Bubble Sort otimizado	$\Theta(n)$	$\Theta(n^2)$	$\Theta(n^2)$
Selection Sort	$\Theta(n^2)$	$\Theta(n^2)$	$\Theta(n^2)$
Insertion Sort	$\Theta(n)$	$\Theta(n^2)$	$\Theta(n^2)$
Merge Sort	$\Theta(n \log n)$	$\Theta(n \log n)$	$\Theta(n \log n)$
Quick Sort clássico	$\Theta(n \log n)$	$\Theta(n \log n)$	$\Theta(n^2)$
Heap Sort	$\Theta(n \log n)$	$\Theta(n \log n)$	$\Theta(n \log n)$

A tabela considera as implementações convencionais discutidas nesta aula.

Para o Bubble Sort, considera-se a versão com interrupção antecipada. O caso médio do Quick Sort pressupõe condições usuais sobre a entrada e a escolha do pivô.

Comparação de memória e estabilidade

Algoritmo	Espaço auxiliar típico	*In-place*?	Estável?
Bubble Sort	$\Theta(1)$	Sim	Sim
Selection Sort	$\Theta(1)$	Sim	Não
Insertion Sort	$\Theta(1)$	Sim	Sim
Merge Sort	$\Theta(n)$	Não, na versão tradicional	Sim
Quick Sort	Pilha de recursão; média $\Theta(\log n)$	Geralmente sim	Não
Heap Sort	$\Theta(1)$	Sim	Não

No **Quick Sort**, o espaço da pilha pode chegar a **O(n)** no pior caso, dependendo da implementação e da estratégia de recursão.

Ibmec
PROTAGONISTAS PARA O MUNDO

Comparação de memória e estabilidade

Ponto de atenção:

a tabela não deve ser interpretada como uma propriedade imutável de qualquer código que receba o nome de um algoritmo.

Implementações específicas podem alterar estabilidade, uso de memória e comportamento operacional.

Por que melhor caso, pior caso e caso médio importam?

Melhor caso — já ordenado

1	2	3	4	5

Poucos deslocamentos: $\Theta(n)$.

Caso intermediário — parcialmente desordenado

2	4	1	5	3

O custo depende de quantos elementos precisam ser deslocados.

Pior caso — ordem inversa

5	4	3	2	1

Muitos deslocamentos: $\Theta(n^2)$.

Por que melhor caso, pior caso e caso médio importam?

O caso médio não é simplesmente o ponto médio entre o melhor e o pior caso.
Ele depende de um modelo de probabilidade para as entradas.
Na análise teórica, normalmente especificamos as hipóteses usadas para obter o custo médio.

Como escolher um algoritmo de ordenação?

Não existe um algoritmo universalmente melhor para todas as situações

- **Entrada pequena ou quase ordenada**

Insertion Sort pode ser uma excelente escolha, por sua simplicidade e desempenho linear no melhor caso.

- **Garantia de tempo no pior caso**

Merge Sort ou **Heap Sort** oferecem **Θ(nlog n)** no pior caso nas versões convencionais.

- **Preservar a ordem de registros com chaves iguais**

Prefira um algoritmo estável, como **Merge Sort** ou **Insertion Sort** nas implementações apresentadas.

- **Pouca memória auxiliar**

Heap Sort é uma opção quando também se deseja garantia de **Θ(nlog n)** no pior caso.

Também é necessário considerar a linguagem, a biblioteca utilizada, a distribuição dos dados e o custo das operações de comparação e movimentação.

Referencias bibliográficas

- SERPA, Matheus S.; RODRIGUES, Thiago N.; ALVES, Ítalo C.; et al Análise de Algoritmos 1. ed. Porto Alegre: SAGAH 2021
- CORMEN, Thomas H.; LEISERSON, Charles E.; Ronald L. Rivest; et al. Algoritmos 4. ed. Rio de Janeiro: GEN LTC2024
- RODRIGUES, Thiago N.; SILVA, Cristiane; HORTA, Gustavo L.; et alTeoria dos grafos e análise de algoritmos 1 ed. Porto Alegre: SAGAH 2022



www.ibmec.br

/ibmec

ibmec

@ibmec_oficial

ibmec

Ibmec

PROTAGONISTAS PARA O MUNDO